BenchPRS: A benchmarking platform for machine learning-based polygenic score model development

license MIT python 3.8+ nextflow 21.04+

A state-of-the-art pipeline for predicting Polygenic Risk Scores (PRS) from genotype data using established PRS methods, PyTorch, sklearn, and Nextflow.

🚀 Features

- **Deep Learning Models**: Multiple architectures (MLP, CNN, Transformer, Attention-based, LSTM, Bayesian NN)
- **Machine Learning Models**: SVM, XGBoost, Random Forests, ElasticNet, and more
- **Established PRS Methods**: Comprehensive collection of proven methodologies
- **Efficient Data Processing**: Handles large-scale PLINK format genotype data
- **Memory Efficient**: Lazy loading with HDF5 for datasets that don't fit in memory
- **Hyperparameter Optimization**: Automated tuning using Optuna
- **Robust Validation**: K-fold cross-validation for reliable evaluation
- **Data Augmentation**: SNP dropout, noise injection, and region shuffling
- **Comprehensive Logging**: Integration with Weights & Biases
- **Rich Visualizations**: Interactive reports and performance metrics

📁 Directory Structure

```
nextflow_pipeline_prs/
├── LICENSE
├── README.md
├── bin/                          # Executable scripts
├── config/                       # Configuration files
│   ├── params.yaml
│   └── training_config.yaml
├── data/                         # Data storage
│   ├── qc/                      # Quality control data
│   ├── raw/                     # Raw input data
│   ├── results/                 # Analysis results
│   └── supplement_data/         # Supplementary datasets
├── models/                       # Model implementations
│   ├── attention_model.py
│   ├── bayesian_model.py
│   ├── cnn_model.py
│   ├── ensemble_model.py
│   ├── mlp_model.py
│   ├── models.py
│   └── transformer_model.py
├── modules/local/               # Local Nextflow modules
├── nextflow.config              # Nextflow configuration
├── out/                         # Output directory
```

```
├── requirements.txt             # Python dependencies
├── run_pipelines.sh            # Main execution script
├── wandb/                       # Weights & Biases logs
└── workflows/                   # Nextflow workflows
    ├── dl_prs.nf
    ├── prs_models_pipeline.nf
    ├── qc_pipeline.nf
    └── sklearn_pipeline.nf
```

🛠️ Installation

Prerequisites

- **Nextflow** (version 21.04+)
- **Python** 3.8+
- **CUDA** 11.0+ (for GPU support, optional)

R Dependencies

Install required R packages:

```
install.packages(c("optparse", "bigsnpr", "tidyr", "ggplot", "Matrix",
                   "data", "fmsb", "devtools", "tidyverse", "magrittr",
                   "bigreadr", "argparser", "stringr",
                   "ggplot2", "cowplot", "bit64", "data.table",
                   , "fmsb"))

install.packages(c())
```

Python Dependencies

Install required Python packages:

```
pip install -r requirements.txt
```

For **GPU support** with PyTorch:

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
```

For **CPU only**:

```
pip install torch torchvision torchaudio
```

Weights & Biases Setup

To use W&B logging (recommended):

```
wandb login
```

PRS Methods Setup

Follow the instructions for each PRS method on their respective repositories:

Method	Repository
LDpred-2	Tutorial
PRSice	Tutorial
lassosum	Tutorial
lassosum2	Documentation
PRSet	Quick Start
PRScs	GitHub
PRScsx	GitHub
SBayesRC	GitHub
Plink1.9	Documentation
SCT	Tutorial

> **Note**: Supplemental data needs to be downloaded as explained for each method and placed into the `supplement_data` folder. LD-specific data should be placed in the `LD` subfolder.

📚 **Additional Resources**:

GWAS Tutorial Open Dataset

🚦 Usage

1. Prepare Your Data

Place your PLINK files in the `data/` directory inside a folder with the same prefix:

- `genotypes.bed` - Binary genotype file
- `genotypes.bim` - SNP information
- `genotypes.fam` - Sample information
- `genotypes.pheno` - Outcome with column named phenotype

Place your summmary statistics file in the `data/supplement_data/sum_stats` file, BenchPRS assumes the following format:

`CHR BP SNP A1 A2 N SE P OR/BETA INFO MAF`

2. Configure the Pipeline

Edit `config/training_config.yaml` to customize:

- Model architectures
- Training parameters
- Hyperparameter search space
- Data augmentation settings

3. Run the Pipelines

Complete pipeline run:

```
bash run_pipelines.sh
```

Individual pipeline runs:

```
# Quality Control
nextflow run workflows/qc_pipeline.nf -params-file
workflows/config/params_qc.yaml

# PRS Models
nextflow run workflows/prs_models_pipeline.nf -params-file
workflows/config/params_prs.yaml

# Deep Learning Models
nextflow run workflows/dl_prs.nf -params-file
workflows/config/dl_config.yaml

# Scikit-learn Models
nextflow run workflows/sklearn_pipeline.nf -params-file
workflows/config/sklearn_config.yaml
```

Resume a previous run:

```
nextflow run main.nf -resume
```

4. Monitor Training

Track experiments in your Weights & Biases dashboard at wandb.ai

> **Note**: Intermediate results are published to Nextflow work directories and can be found through the unique identifier associated with each process.

📊 Output Files

Trained Models

- `results/models/fold_*/best_model_fold_*.pt` - Best model checkpoints
- `results/models/fold_*/training_history_fold_*.json` - Training metrics

Evaluation Reports

- `results/evaluation/final_evaluation_report.html` - Interactive HTML report
- `results/evaluation/ensemble_predictions.csv` - PRS predictions with risk categories
- `results/evaluation/model_comparison.json` - Performance metrics

Visualizations

- `results/visualizations/data_overview.png` - Data quality plots
- `results/visualizations/interactive_viz.html` - Interactive visualizations

📈 Performance Metrics

The pipeline evaluates models using PRS-specific metrics:

- **R^2 Score**: Variance explained
- **AUC-ROC**: Classification evaluation
- **Pearson/Spearman Correlation**: Linear/rank correlation
- **Risk Stratification**: Odds ratios, risk ratios
- **Decile Analysis**: Performance across risk deciles

🔧 Running Individual Scripts

You can run pipeline components individually:

Convert PLINK to HDF5

```
python bin/plink_converter.py \
    --plink_prefix data/genotypes \
    --output_h5 results/genotypes.h5 \
    --output_pheno results/phenotypes.csv \
    --stats_file results/stats.json
```

Split Data

```
python bin/data_splitter.py \
    --genotype_file results/genotypes.h5 \
    --phenotype_file results/phenotypes.csv \
    --output_splits results/splits.json \
    --output_indices results/indices.npz
```

Train a Model

```
python bin/train_model.py \
    --genotype_file results/genotypes.h5 \
    --phenotype_file results/phenotypes.csv \
    --indices_file results/indices.npz \
    --params_file config/training_config.yaml \
    --fold 0 \
    --output_model results/model.pt \
    --output_metrics results/metrics.json \
    --output_log results/log.csv
```

🔍 Troubleshooting

Memory Issues

- Reduce `batch_size` in configuration
- Increase `cache_size` in dataset settings
- Use data subset for testing

GPU Issues

- **Check CUDA availability**:

  ```
  python -c "import torch; print(torch.cuda.is_available())"
  ```

- **Run on CPU**: Set `device: cpu` in config

Nextflow Issues

- **Clear cache**: `nextflow clean -f`
- **Check logs**: `cat .nextflow.log`
- **Update Nextflow**: `nextflow self-update`

📄 License

This project is licensed under the MIT License - see the LICENSE file for details.

🤝 Contributing

Contributions are welcome! Please feel free to submit a Pull Request.

📧 Support

If you encounter any issues or have questions, please open an issue on GitHub.